

EXAM IN: STA510 STATISTICAL MODELING AND SIMULATION

DURATION: 4 HOURS

DATE: FEBRUARY 13th, 2023

PERMITTED AIDS: Approved simple calculator

THE EXAM CONSISTS OF 5 PROBLEMS ON 8 PAGES, 9 PAGES OF ENCLOSURES.

COURSE RESPONSIBLE: Tore Selland Kleppe

PHONE: 51 83 17 17

All points a), b), c) and so on are weighted equally. There are a total of 22 such points.

Problem 1

Consider a random variable X with probability density function

$$f(x) = \frac{3\sqrt{x}}{16}, 0 \leq x \leq 4.$$

- Verify that $f(x)$ is a proper probability density function.
- Find $E(X)$.
- Find $E(\sqrt{X})$ and $Var(\sqrt{X})$.
- Show that the cumulative distribution function of X is given by $F(x) = \frac{x^{3/2}}{8}$.
- Find the inverse cumulative distribution function, and explain with words how you would simulate n random numbers with density $f(x)$ using the inverse transform method.

Now consider the following R-functions:

```
p1.inv.1 <- function(n){
  return((3/16)*sqrt(runif(n)))
}
p1.inv.2 <- function(n){
  return(4*runif(n,max=4)^(2/3))
}
p1.inv.3 <- function(n){
  return(4*runif(n)^(2/3))
}
p1.inv.4 <- function(n){
  return((1/8)*runif(n)^(3/2))
}
```

- Which of the above functions correctly simulates random numbers with density $f(x)$? Argue for why the remaining functions produce wrong results.

Now you consider simulating random variables with density $f(x)$ using the accept-reject (AR) algorithm. Specifically, we consider two different proposal distributions with densities $g_1(x)$ and $g_2(x)$ where

$$g_1(x) = \begin{cases} 1/4 & \text{when } 0 \leq x \leq 4, \\ 0 & \text{otherwise,} \end{cases}$$

$$g_2(x) = \begin{cases} C\sqrt{x} \exp(-x/20) & \text{when } 0 \leq x \leq 4, \\ 0 & \text{otherwise,} \end{cases}$$

where $C \approx 0.2111142793$. Note that $g_1(x)$ is recognized to be the density of a $Uniform(0, 4)$ random variable and that $g_2(x)$ is a “truncated gamma” distribution which allows easy simulation.

- g) From a performance perspective (assuming here that it is equally costly to simulate random numbers from either density), which proposal distribution would you choose for an AR-algorithm? Why?

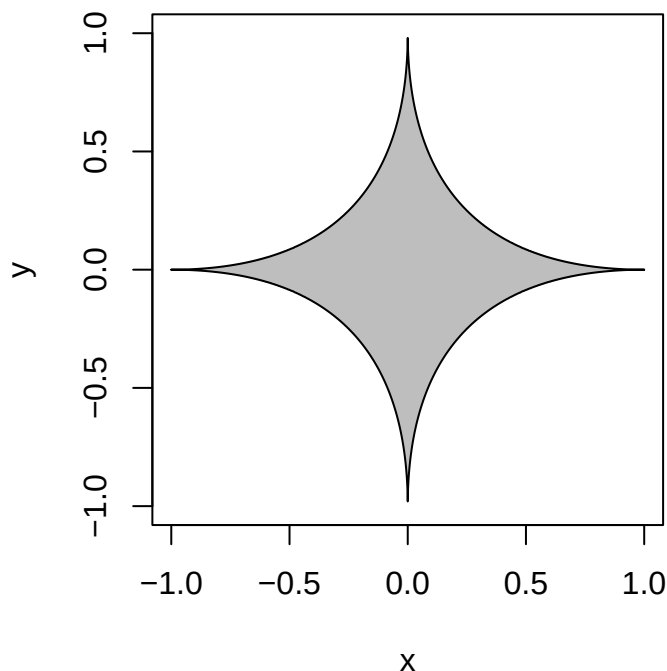
Problem 2

This problem is similar to Problem 2 on Mandatory assignment 3.

We consider the “ $L^{1/2}$ -unit circle” in two dimensions, defined to be all points (x, y) so that

$$\left(\sqrt{|x|} + \sqrt{|y|}\right)^2 \leq 1$$

The points that obey the above inequality are indicated with gray color in the figure below:



We wish to estimate the area of $L^{1/2}$ -unit circle, say A , which may be expressed as

$$A = \int_{-1}^1 \int_{-1}^1 \mathbf{1}(x, y) dx dy \quad \text{where } \mathbf{1}(x, y) = \begin{cases} 1 & \text{if } \left(\sqrt{|x|} + \sqrt{|y|}\right)^2 \leq 1, \\ 0 & \text{otherwise.} \end{cases}$$

A Monte Carlo estimator of A is given by

$$\hat{A} = \frac{4}{n} \sum_{i=1}^n \mathbf{1}(u_i, v_i)$$

where $u_i \sim \text{iid } Uniform(-1, 1)$ and $v_i \sim \text{iid } Uniform(-1, 1)$, and all u_i s and v_i s are independent.

Consider the following R-code

```
n <- 1000000
u <- runif(n,min=-1,max=1)
v <- runif(n,min=-1,max=1)

indicator <- (sqrt(abs(u))+sqrt(abs(v)))^2 <= 1.0
print(median(indicator))

## [1] 0

print(mean(indicator))

## [1] 0.166846

print(sd(indicator))

## [1] 0.3728385

print(quantile(indicator))

##    0%   25%   50%   75%  100%
##    0    0    0    0    1
```

- a) Based on the above information, provide a point estimate of A and an approximate 95% confidence interval for A .

It can be shown that

$$\sum_{i=1}^n \mathbf{1}(u_i, v_i) \sim \text{Binomial}\left(\frac{1}{6}, n\right).$$

- b) Find the exact number of simulations n needed in order for the standard deviation of $\hat{A}$ to be smaller than 0.01.
- c) For $n = 1000$, calculate (approximately) $P(\hat{A} < 0.67)$ without using the simulation results.

Problem 3

Consider the probability density

$$f(x) = \begin{cases} C \exp(-x^2/2) & \text{if } -2 \leq x \leq 2, \\ 0 & \text{otherwise,} \end{cases}$$

where $C \approx 0.4179595500$.

- a) Describe, with words, an algorithm for generating iid random numbers from the above probability density function. Any constants etc needed to implement your algorithm should be given with numerical values.

Below is an algorithm for generating dependent random variables from the distribution with density $f(x)$ above:

```
1 n <- 100000
2 x <- numeric(n)
3 x[1] <- 0.0
4 C <- 0.4179595500
5 wt.old <- C*exp(-0.5*x[1]^2)/dunif(x[1],min=-2,max=2)
6 for(i in 2:n){
7   x.prop <- runif(1,min=-2,max=2)
8   wt.prop <- C*exp(-0.5*x.prop^2)/dunif(x.prop,min=-2,max=2)
9   if(runif(1)<min(1,wt.prop/wt.old)){
10    x[i] <- x.prop
11    wt.old <- wt.prop
12  } else {
13    x[i] <- x[i-1]
14  }
15 }
```

- b) Which algorithm has been implemented above? Describe the choices made when designing the algorithm.

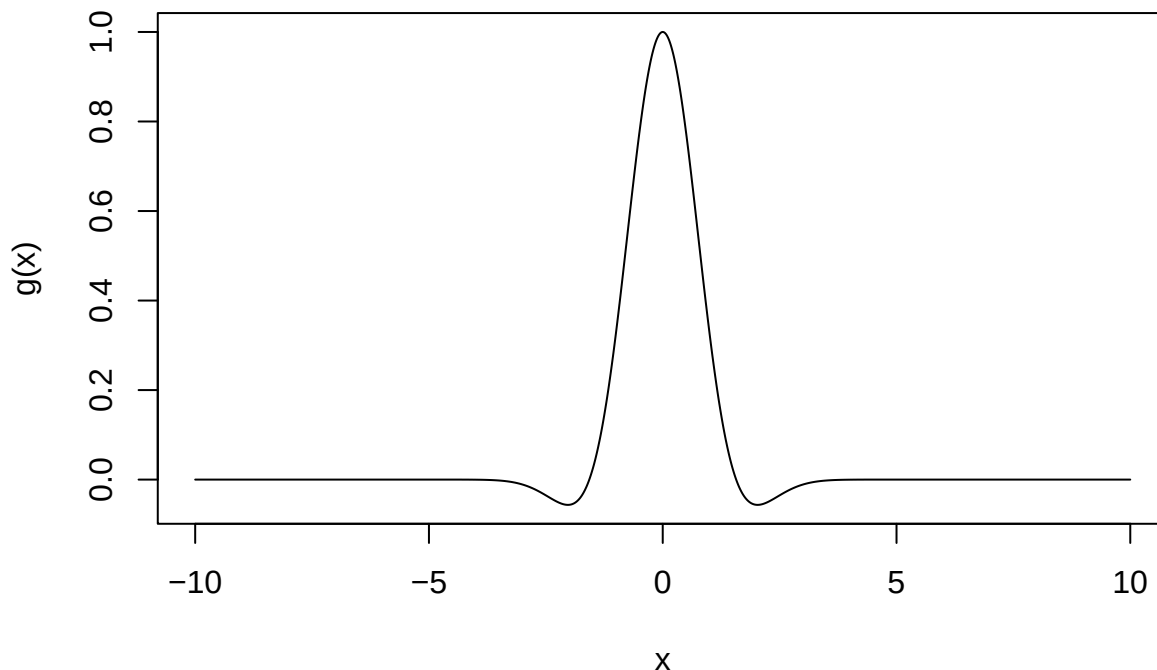
Problem 4

Consider the integrals

$$A = \int_{-\infty}^{\infty} \exp\left(-\frac{x^2}{2}\right) \cos(x) dx,$$

$$B = \int_{-\pi/2}^{\pi/2} \exp\left(-\frac{x^2}{2}\right) \cos(x) dx.$$

The integrand $g(x) = \exp\left(-\frac{x^2}{2}\right) \cos(x)$ is plotted below:



- a) Write down an expression for a Monte Carlo estimator of A based on n random draws from a $N(0, 1)$ distribution.
- b) Write down an expression for a Monte Carlo estimator of B based on n random draws from a $N(0, 1)$ distribution.
- c) Write down an expression for a Monte Carlo estimator of B based on n random draws from a $Uniform(-\pi/2, \pi/2)$ distribution.

The Laplace(0,2) distribution has probability density function $p(x) = \exp(-|x|/2)/4$.

- d) Write down an expression for a Monte Carlo estimator of A based on n random draws from a Laplace(0,2) distribution.
- e) Write down an expression for a Monte Carlo estimator of B based on n random draws from a Laplace(0,2) distribution.
- f) Write down an expression for an antithetic Monte Carlo estimator for B that evaluates the integrand n times and uses random draws from a $Uniform(-\pi/2, \pi/2)$ -distribution. Do you expect this estimator to work better or worse than the one in c)?

Problem 5

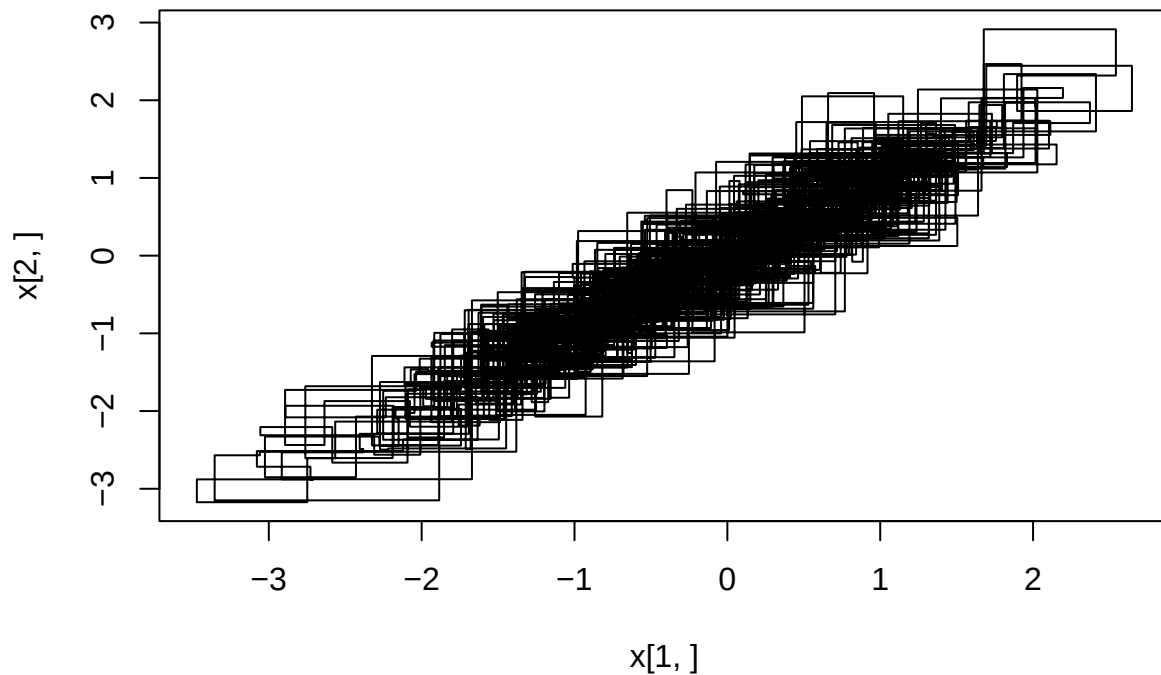
In this problem we consider different types of algorithms for simulating from a bivariate normal distribution¹ with mean vector μ and covariance matrix Σ given by

$$\mu = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \Sigma = \begin{pmatrix} 1 & 0.9 \\ 0.9 & 1 \end{pmatrix}.$$

You may take as granted that all of the algorithms produce correct results, and they all fill the random vectors into the columns of a matrix named $\mathbf{x}$.

The first algorithm, and a plot of output are given by:

```
n <- 1000
rho <- 0.9
x1 <- 0.0
x2 <- 0.0
x <- c(x1,x2)
for(i in 1:n){
  x1 <- rnorm(1,mean=rho*x2,sd=sqrt(1.0-rho^2))
  x <- cbind(x,c(x1,x2))
  x2 <- rnorm(1,mean=rho*x1,sd=sqrt(1.0-rho^2))
  x <- cbind(x,c(x1,x2))
}
plot(x[1,],x[2,],type="l")
```

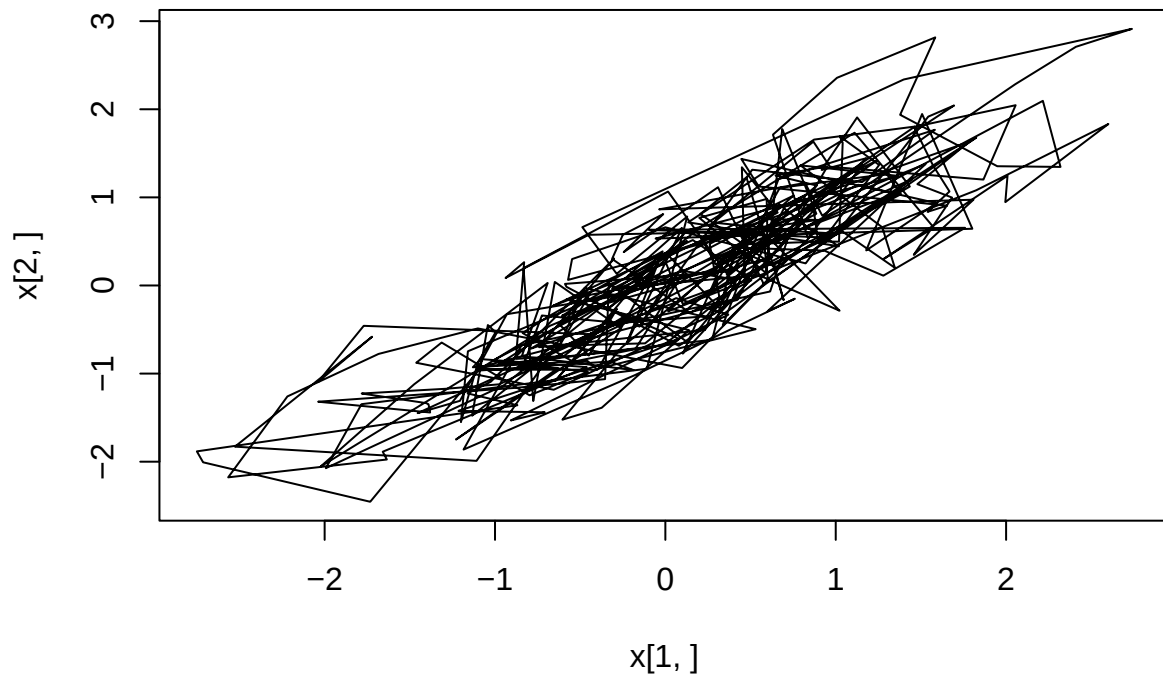


a) Which type of algorithm is this? Why?

¹I.e. a multivariate normal distribution with dimension 2.

The next algorithm is given by:

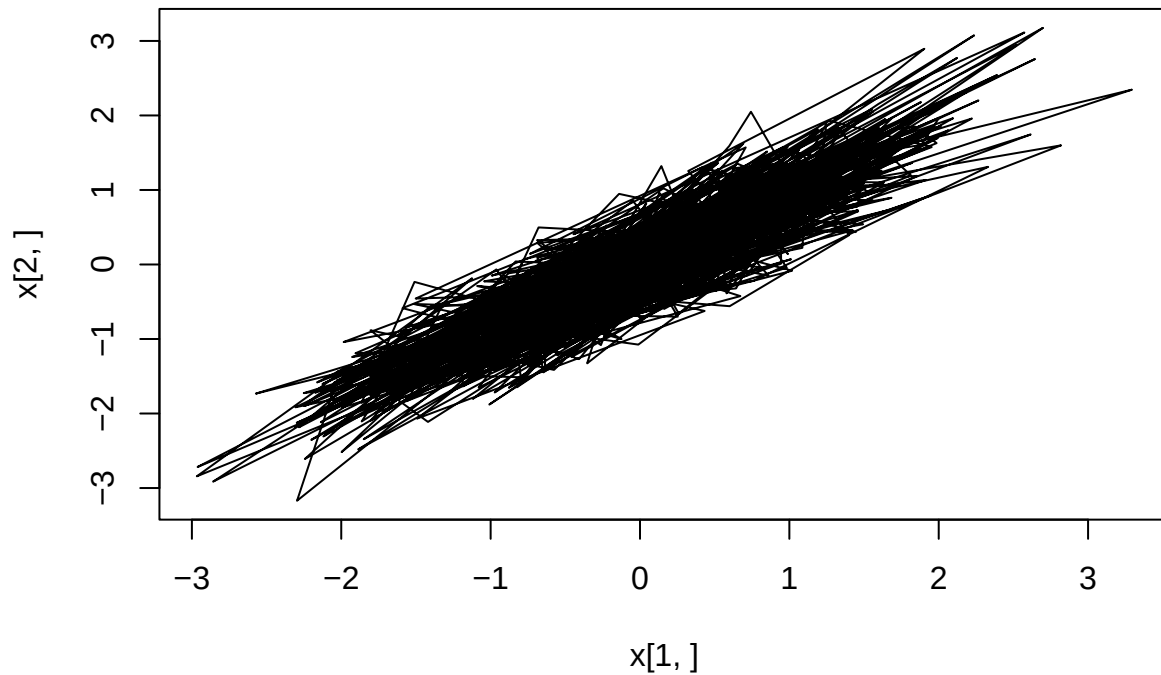
```
n <- 1000
x <- matrix(0.0,2,n)
x[,1] <- c(0,0)
Sigma <- matrix(c(1,0.9,0.9,1),2,2)
for(i in 2:n){
  x.prop <- x[,i-1]+rnorm(2)
  alpha <- min(1.0,mvtnorm::dmvnorm(x.prop,mean=c(0,0),sigma=Sigma)/
              mvtnorm::dmvnorm(x[,i-1],mean=c(0,0),sigma=Sigma))
  if(runif(1)<alpha){
    x[,i] <- x.prop
  } else {
    x[,i] <- x[,i-1]
  }
}
plot(x[,1],x[,2],type="l")
```



b) Which type of algorithm is this? Why?

Now consider:

```
n <- 1000
x <- matrix(0.0,2,n)
L <- t(chol(matrix(c(1,0.9,0.9,1),2,2)))
for(i in 1:n){
  x[,i] <- L%*%rnorm(2)
}
plot(x[1,],x[2,],type="l")
```



- c) Which type of algorithm is this?
- d) Which of the three algorithms above would you prefer? Why?